



Experiment 5

Student Name: Nitin

Branch: CSE

Semester: 6th

Subject Name: System Design

UID: 23BCS12270

Section/Group: KRG 3-A

Date of Performance: 25/02/2026

Subject Code: 23CSH-314

1. Aim: To design and analyze a scalable real-time messenger application (similar to WhatsApp / Facebook Messenger) that allows users to register, send one-to-one and group messages, share media, and receive messages instantly while ensuring high availability, reliability, and low latency using distributed system concepts.

2. Objective:

- To identify functional and non-functional requirements of a messaging system
- To design a high-level architecture (HLD) for a large-scale chat application
- To understand real-time communication using WebSocket
- To apply CAP theorem trade-offs in messaging systems
- To understand message queues, storage, caching, and delivery guarantees
- To study how availability and reliability are prioritized in chat systems

3. Tools:

- Draw.io – System Architecture & HLD Diagram
- MySQL / PostgreSQL – User & group metadata
- NoSQL DB (MongoDB / Cassandra) – Message storage
- Redis – Online presence & caching
- WebSocket Server – Real-time messaging
- Apache Kafka / Message Queue – Asynchronous delivery
- Object Storage (S3 equivalent) – Media files
- Load Balancer – Traffic distribution

4. System Requirements:

A. Functional Requirements:-

- 1) Client should be able to register and login securely
- 2) System should support one-to-one messaging
- 3) System should support group messaging
- 4) System should support text and media messages (image/video/file)
- 5) System should preserve chat history
- 6) Client should see message delivery and read receipts

- 7) Client should view chat list and conversation history
- 8) System should support creating and managing groups

B. Non-Functional Requirements:-

1. Scalability

- Target Users: 1 Billion users
- Avg Messages/User/Day: 100
- Total Messages: 100 Billion/day
- Storage Estimate: ~100 TB messages

System must handle millions of concurrent connections.

2. Availability vs Consistency (CAP Theorem)

Availability >>> Consistency (Eventual Consistency)

Reason:

- Messages must be delivered even during failures
- Slight delay in read receipts is acceptable
- Losing messages is unacceptable

Example:

- Message delayed → Acceptable
- Message lost → Critical failure

3. Latency

Target latency: 200–300 ms

Messages should appear almost instantly.

4. Reliability

- No message loss
- Guaranteed delivery
- Retry & acknowledgment mechanism

5. High Level Design (HLD):

The system follows a microservices-based architecture:

API Gateway

- Authentication, routing, rate limiting
- Entry point for all clients



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

User Service

- Registration & login
- Profile & contacts
- Stored in SQL DB

Messaging Service

- Handles sending/receiving messages
- Stores messages in NoSQL DB
- Maintains conversation threads

Real-Time WebSocket Server

- Persistent connections with clients
- Instant message delivery
- Presence & typing indicators

Group Service

- Group creation & membership
- Group metadata storage

Media Service

- Upload & download media files
- Stores media in object storage

Message Queue (Kafka)

- Asynchronous message delivery
- Decouples sender & receiver
- Ensures reliability

Redis Cache

- Online/offline status
- Recent chats
- Unread counts

Notification Service

- Push notifications for offline users

6. Messaging Workflow:

Step-1: User sends message

Step-2: Client → WebSocket Server

Step-3: Message stored in DB

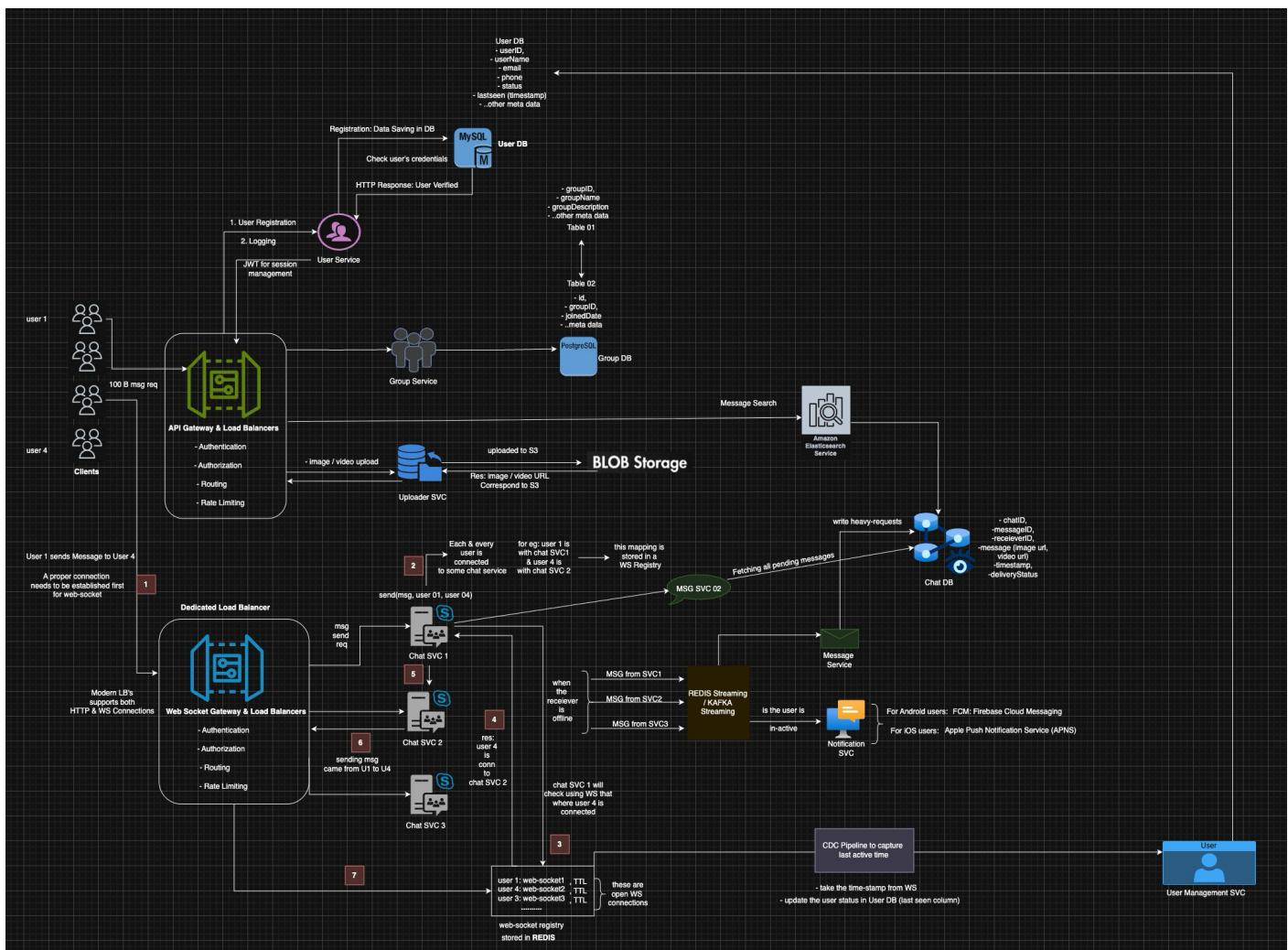
Step-4: Message pushed to receiver if online

Step-5: If offline → Queue + Notification

Step-6: Receiver reads → Read receipt sent

7. Scalability Solution:

- Horizontal scaling of WebSocket servers
- Sharding messages by user ID
- Kafka queues for async delivery
- Redis caching for presence
- NoSQL DB for massive writes
- Load balancer distributes connections
- Stateless microservices





8. Learning Outcomes (What I Have Learnt):

- Understood how real-time messaging apps like WhatsApp are designed
- Learned difference between functional & non-functional requirements
- Gained understanding of WebSocket & real-time communication
- Learned CAP theorem trade-offs in chat systems
- Understood message queues & delivery guarantees
- Learned how to design highly available messaging systems